

# 前言

- 要早做，有意识得经常性得去做  
不要等到最后才去做优化  
不要盲目得去优化，在检测到问题之后再针对的去做
- 尽早得在目标硬件平台做测试
- 项目的每个人员都应该了解基本的性能优化，懂得资源优化的美术人员可以节省大量的优化时间

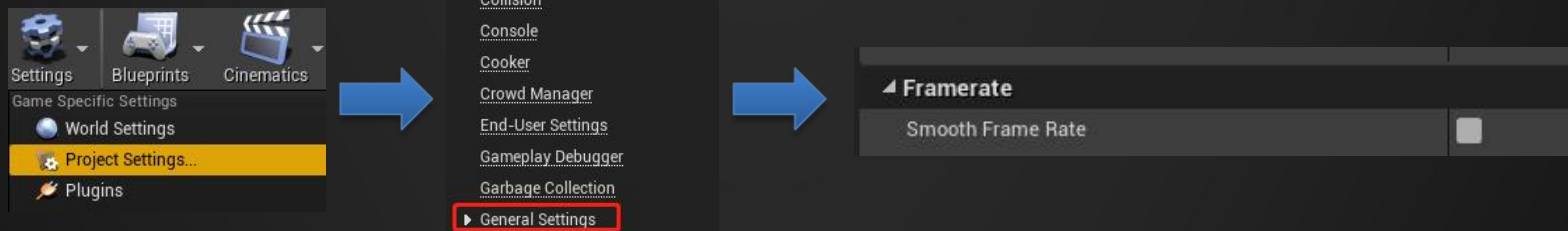
# 优化准备工作

性能分析会有其它的干扰项，为了让数据更加准确，需要做一些准备工作来最小化干扰。

- 关掉一切无用的进程
- 关掉垂直同步（调用控制台命令（小键盘最左边的“~”键）`"r.vsync 0"`）

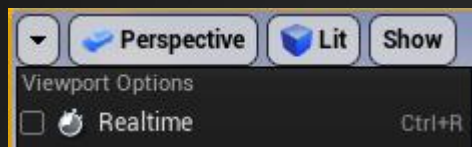


- 关掉帧率平滑

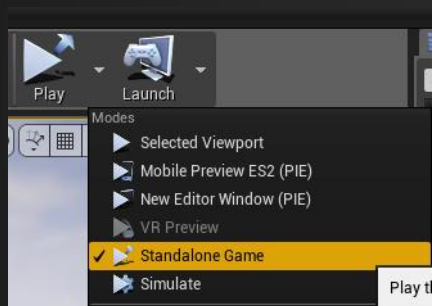


# 优化准备工作

- 关掉Realtime



- 在Standalone模式下运行



- 最小化编辑器

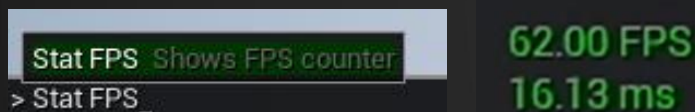
# 找到瓶颈

影响一个游戏帧率有多种因素，优化之前先找到最大瓶颈所在再针对进行优化。

- 使用控制台命令“*r.ScreenPercentage 10*”，如果游戏突然加快说明就是GPU遇到瓶颈（这条命令的作用是修改屏幕分辨率的百分比，*10*代表把分辨率的百分比调成10%，分辨率越低意味着需要渲染的像素点越少，减少GPU的负担，如果此时游戏突然加快，就说明之前的瓶颈在GPU上）

# 找到瓶颈

- 使用控制台命令“*Stat FPS*”打开FPS（Frames Per Second, 每秒刷新的帧数，FPS越高说明屏幕刷新得越快）以及Frame（刷新一帧需要得时间，单位毫秒，Frame越低说明屏幕刷新得越快）。



The screenshot shows a console window with a dark background. On the left, there is a text box containing the command "Stat FPS" and its output "Shows FPS counter". To the right of the text box, the current FPS and Frame values are displayed in green text: "62.00 FPS" and "16.13 ms".

| Metric     | Value |
|------------|-------|
| FPS        | 62.00 |
| Frame (ms) | 16.13 |

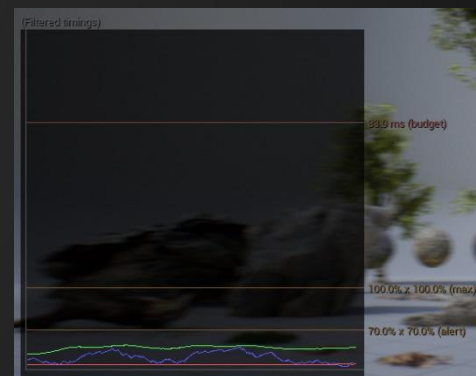
- “*Stat FPS*”只是用作标准参考得数据，并不能用作瓶颈分析。

# 找到瓶颈

- 使用控制台命令“*Stat Unit*”把Frame拆成Game/Draw/GPU三个部分，数值最接近Frame的基本就是瓶颈所在了。

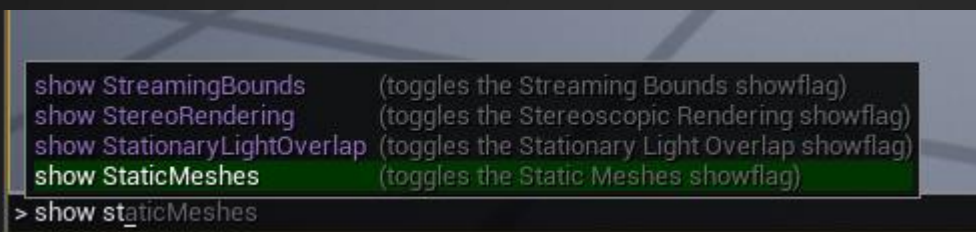
```
Frame: 16.13 ms  
Game: 4.33 ms  
Draw: 5.04 ms  
GPU: 16.13 ms
```

- Frame**: 刷新一帧需要得时间。
- Game**: C++或者蓝图的游戏逻辑消耗。
- Draw**: CPU渲染消耗。
- GPU**: GPU渲染消耗。
- 也可以使用控制台命令“*Stat UnitGraph*”以图表的形式显示数据。



# 找到瓶颈

- 还有一种快速分析的方式就是控制显示的类型  
“*Show xxx*”（比如说“*Show Staticmeshes*”切换静态模型的显示）



- Staticmeshes  
Skeletameshes  
Particles  
Translucency  
等等

# 瓶颈处理

找到瓶颈之后要知道各自瓶颈的影响因素，再针对各自的因素考虑解决办法。

- Game—代码或者蓝图
- CPU —绘制指令（DrawCall），剔除
- GPU —材质，重复绘制（OverDraw），灯光



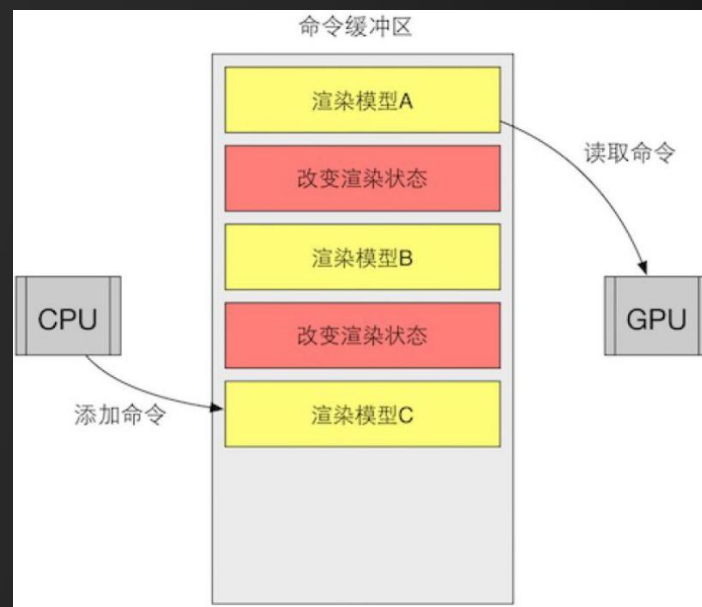
# 瓶颈处理 · Draw Call

什么是Draw Call?

- Draw Call是CPU对GPU的一道命令,CPU跟GPU的通信分为三个阶段, Draw Call处于第三阶段。

- Draw Call实质

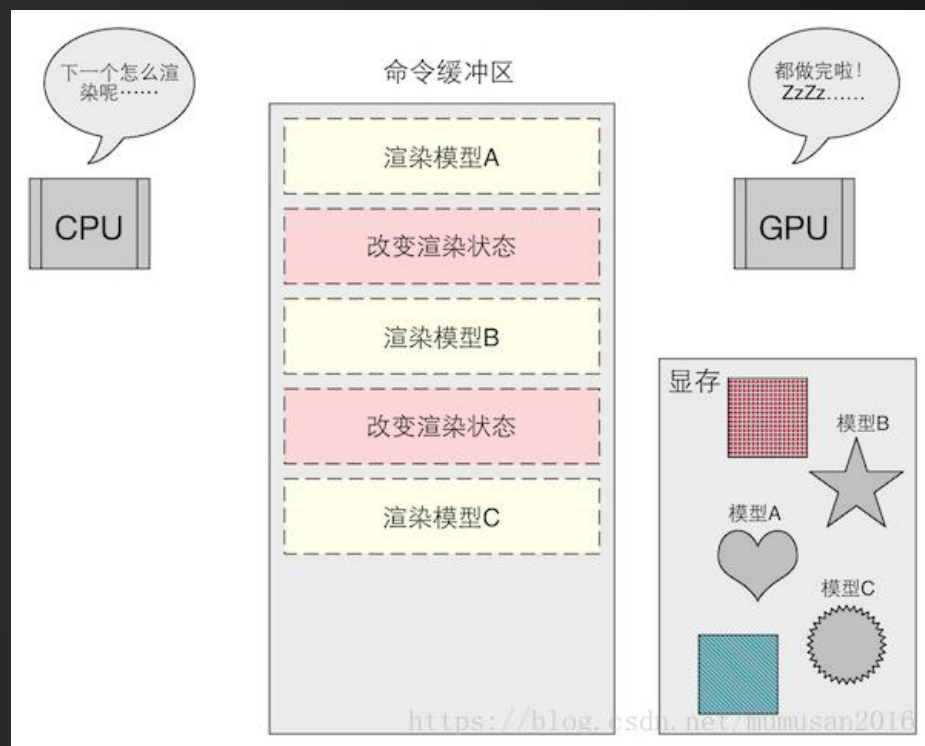
CPU跟GPU是并行运行的, 而保证CPU和GPU可以并行工作的解决方法是: 创建命令缓冲区 ,CPU发布命令,GPU在完成上一次渲染任务之后会从中再取出命令并且执行。



# 瓶颈处理 · Draw Call

为什么Draw Call多了会影响帧率？

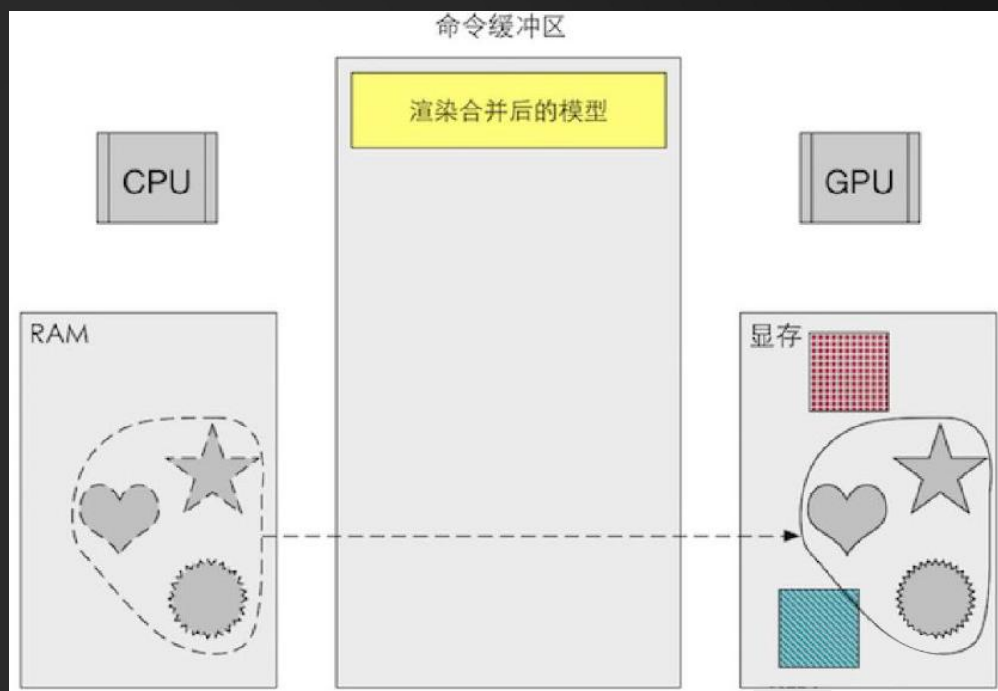
- 在每次调用Draw Call之前，CPU需要向GPU发送很多内容，包括数据，状态等，在这一阶段，CPU需要完成很多工作，例如检查渲染状态等。而一旦CPU完成了准备工作，GPU开始本次的渲染。由于GPU的多核性，GPU的渲染能力是很强的，渲染300个和3000个三角模型通常没有什么区别，因此渲染速度往往快于CPU。如果Draw Call过多，CPU就会把大量时间花费在提交Draw Call的命令上，造成CPU过载。
- 因此造成Draw Call性能问题的元凶是CPU。



# 瓶颈处理 · Draw Call

## 怎么减少Draw Call?

- 提交大量很小的Draw Call会造成CPU的瓶颈，CPU把时间都花费在准备Draw Call上了。那么，一个显然的优化想法就是把很小的Draw Call合并成一个大的Draw Call，这就是批处理的思想。
- 但要注意的是，使用批处理的模型将会使用同一种渲染状态。也就是说，如果模型之间使用不同的渲染状态，那么使用批处理技术将没有意义。



# 瓶颈处理 · Draw Call

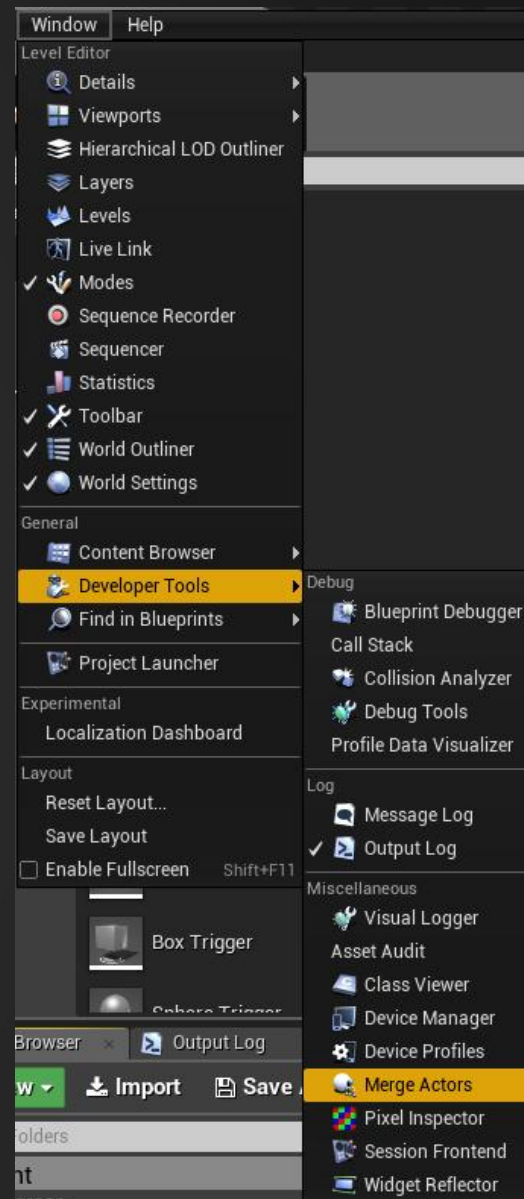
关于Draw Call优化的总结。

- 避免使用大量小模型，尽量把具有相同材质的模型合并成一个大模型。
- 减少单个模型材质ID，能用一个材质解决就不要出现第二个材质，因为不同材质使用不同的渲染状态。
- 尽量在不同模型之间共用一个材质。
- 需要注意的是，由于我们需要在CPU的内存中合并模型，而这个过程是需要消耗时间的。因此，批处理更适合静态的模型，例如不会移动的房子，石头等，对于这些静态物体只需要合并一次就可以了。当然，我们也可以对动态物体进行批处理，但是，因为这些物体是不断运动的，因此每一帧都需要重新进行合并然后再发送给GPU，这样反而不合算了。

# 瓶颈处理 · Draw Call

## UE4批处理的方法

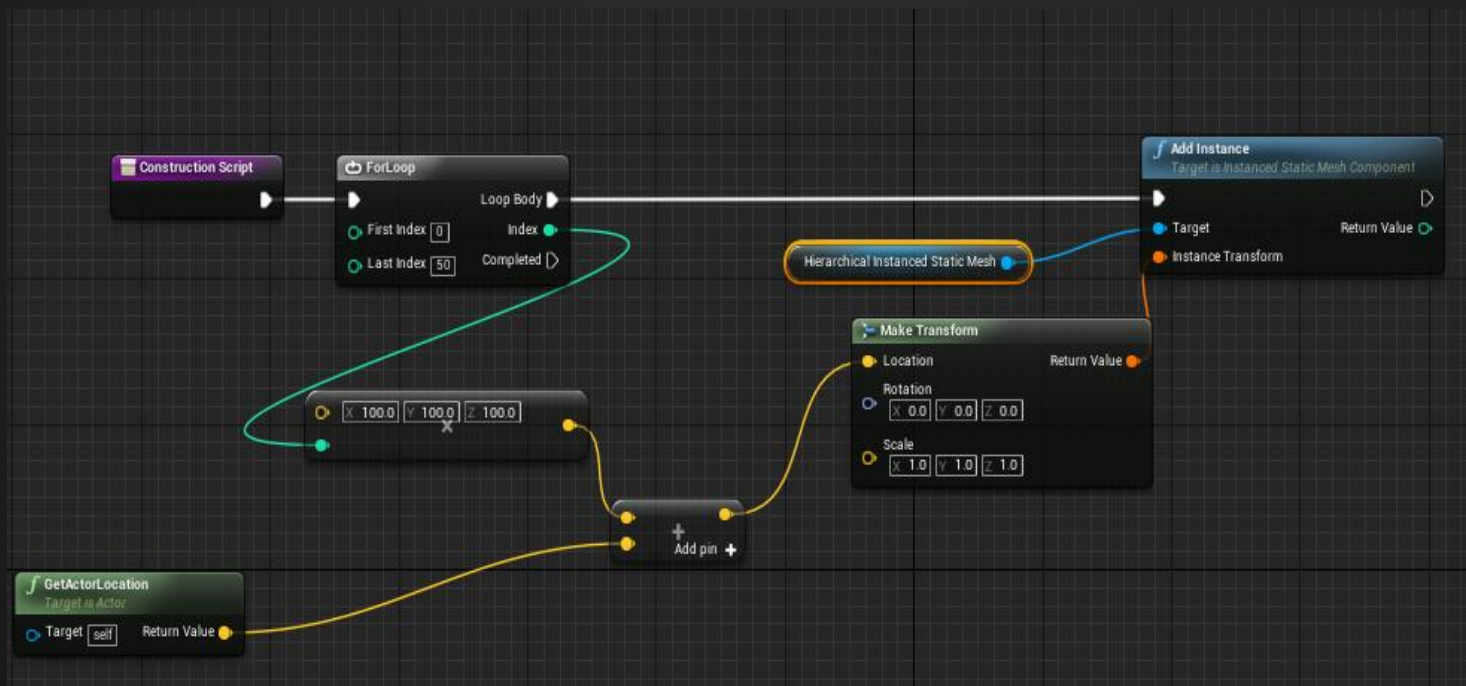
- 使用开发者工具中的Merge Actors工具。



# 瓶颈处理 · Draw Call

## UE4批处理的方法

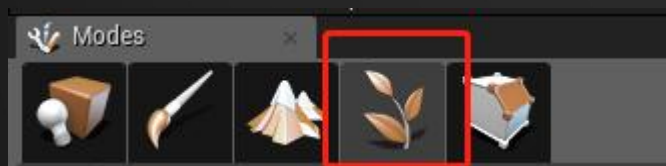
- 使用蓝图中的Instanced Static Meshes功能，作用是指定一个静态模型拷贝出大量的实例，最后只算一个模型的Draw Call。



# 瓶颈处理 · Draw Call

## UE4批处理的方法

- 使用Foliage系统。
- 适用于植被系统，例如树，草地等等。
- 自由度高，可以自由控制植被的位置，疏密，大小等等。
- 跟Instanced Static Meshes一样，最后形成是单个的Instanced Actor，只算一个模型的Draw Call。

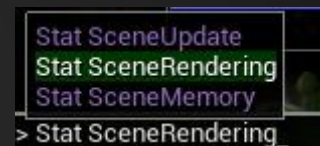




# 瓶颈处理 · Draw Call

Draw Call次数的查看方法。

- 使用控制台命令“*Stat SceneRendering*”。



| Scene Rendering [STATGROUP: SceneRendering]                           | CallCount | InclusiveAvg | InclusiveMax | ExclusiveAvg | ExclusiveMax |
|-----------------------------------------------------------------------|-----------|--------------|--------------|--------------|--------------|
| RenderViewFamily                                                      | 1         | 2.08 ms      | 2.59 ms      | 0.07 ms      | 0.08 ms      |
| DeferredShadingSceneRenderer.Lighting                                 | 1         | 0.40 ms      | 0.61 ms      | 0.08 ms      | 0.14 ms      |
| Base pass drawing                                                     | 1         | 0.31 ms      | 0.45 ms      | 0.01 ms      | 0.01 ms      |
| InitViews                                                             | 1         | 0.27 ms      | 0.32 ms      | 0.01 ms      | 0.01 ms      |
| FinishRenderViewTarget                                                | 1         | 0.20 ms      | 0.34 ms      | 0.00 ms      | 0.00 ms      |
| StaticDrawList drawing                                                | 1         | 0.27 ms      | 0.39 ms      | 0.23 ms      | 0.34 ms      |
| Depth drawing                                                         | 1         | 0.22 ms      | 0.28 ms      | 0.19 ms      | 0.24 ms      |
| Lighting drawing                                                      | 1         | 0.22 ms      | 0.34 ms      | 0.00 ms      | 0.00 ms      |
| InitViewsPossiblyAfterPrepass                                         | 1         | 0.15 ms      | 0.19 ms      | 0.01 ms      | 0.01 ms      |
| Dynamic shadow setup                                                  | 1         | 0.11 ms      | 0.14 ms      | 0.03 ms      | 0.04 ms      |
| Proj Shadow drawing                                                   | 1         | 0.09 ms      | 0.15 ms      | 0.01 ms      | 0.02 ms      |
| DeferredShadingSceneRenderer.AfterBasePass                            | 1         | 0.06 ms      | 0.09 ms      | 0.05 ms      | 0.08 ms      |
| Dynamic Primitive drawing                                             | 1         | 0.04 ms      | 0.06 ms      | 0.03 ms      | 0.05 ms      |
| Translucency drawing                                                  | 1         | 0.03 ms      | 0.03 ms      | 0.02 ms      | 0.03 ms      |
| DeferredShadingSceneRenderer.Render.Init                              | 1         | 0.02 ms      | 0.03 ms      | 0.02 ms      | 0.03 ms      |
| Begin Occlusion Tests                                                 | 1         | 0.02 ms      | 0.02 ms      | 0.02 ms      | 0.02 ms      |
| DeferredShadingSceneRenderer.DBUFFER                                  | 1         | 0.01 ms      | 0.01 ms      | 0.01 ms      | 0.01 ms      |
| RenderVelocities                                                      | 1         | 0.01 ms      | 0.01 ms      | 0.01 ms      | 0.01 ms      |
| ...dSceneRenderer.FXSystem.PostRenderOpaque                           | 1         | 0.01 ms      | 0.01 ms      | 0.00 ms      | 0.00 ms      |
| ...dShadingSceneRenderer.FXSystem.PreRender                           | 1         | 0.01 ms      | 0.01 ms      | 0.00 ms      | 0.00 ms      |
| ...dingSceneRenderer.SetAndClearViewGBuffer                           | 1         | 0.01 ms      | 0.01 ms      | 0.01 ms      | 0.01 ms      |
| ...hadingSceneRenderer.DistanceFieldAO.Init                           | 1         | 0.01 ms      | 0.01 ms      | 0.01 ms      | 0.01 ms      |
| ...dSceneRenderer.RenderLightShaftOcclusion                           | 1         | 0.00 ms      | 0.00 ms      | 0.00 ms      | 0.00 ms      |
| ...adingSceneRenderer.RenderLightShaftBloom                           | 1         | 0.00 ms      | 0.00 ms      | 0.00 ms      | 0.00 ms      |
| ...ndSceneRenderer.Render.ServoLocalQueue                             | 25        | 0.00 ms      | 0.01 ms      | 0.00 ms      | 0.01 ms      |
| [11 more stats. Use the stats.MaxPerGroup CVar to increase the limit] |           |              |              |              |              |
| Counters:                                                             | Averages: | MSK:         |              |              |              |
| Mesh draw calls                                                       | 430.00    | 430.00       |              |              |              |
| Static list draw calls                                                | 416.00    | 416.00       |              |              |              |
| Present time                                                          | 8.67 ms   | 8.16 ms      |              |              |              |
| Lights in scene                                                       |           | 24.00        |              |              |              |
| Lights using light shafts                                             |           |              |              |              |              |
| Translucency GPU Time (MS)                                            | 0.00      | 0.00         |              |              |              |

- 图表下面显示的就是Draw Call数量。



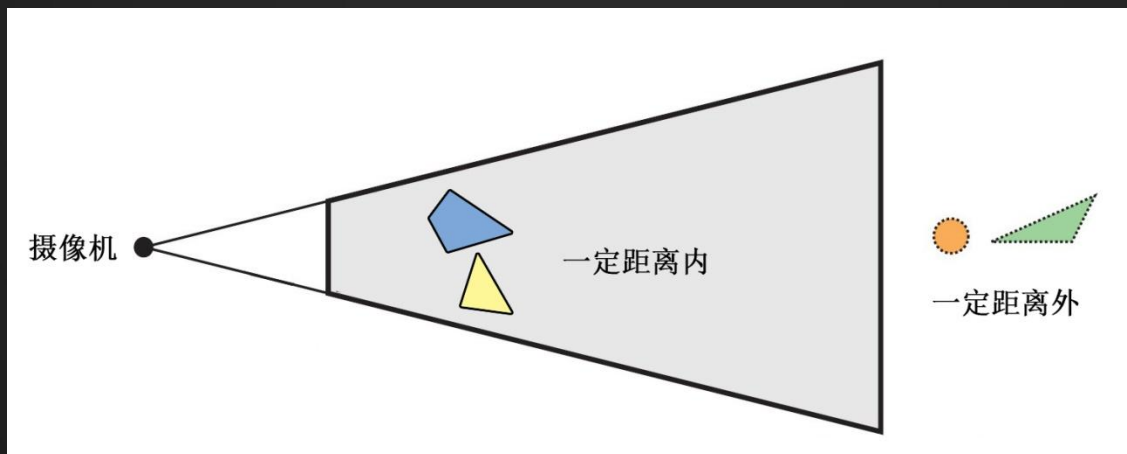
# 瓶颈处理 · 剔除 (Culling)

- 所谓剔除，简单点来说就是将不需要渲染的东西给排除掉，减轻渲染压力。
- 剔除有多种方法，这里主要讲距离剔除、视锥体剔除、遮挡剔除跟Early Z Pass 。

# 瓶颈处理 · 剔除 (Culling)

## 距离剔除(Distance Culling)

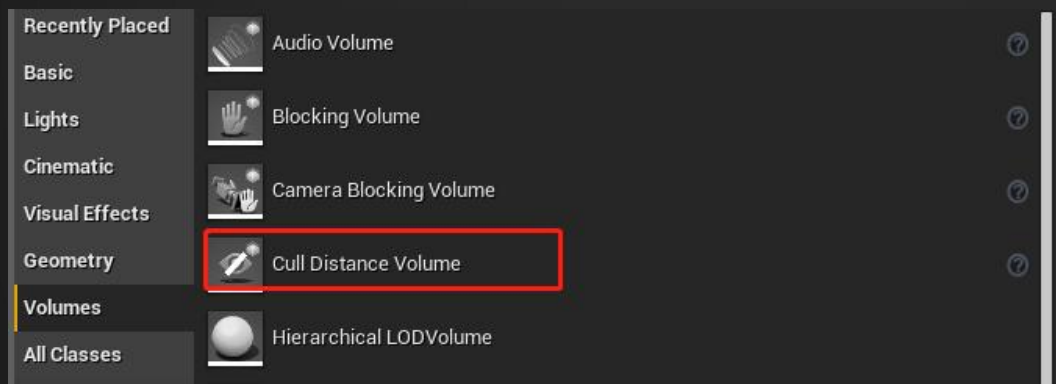
- 跟LOD类似的优化思路，距离屏幕远的、不重要的物体给剔除掉，不渲染出来。



# 瓶颈处理 · 剔除 (Culling)

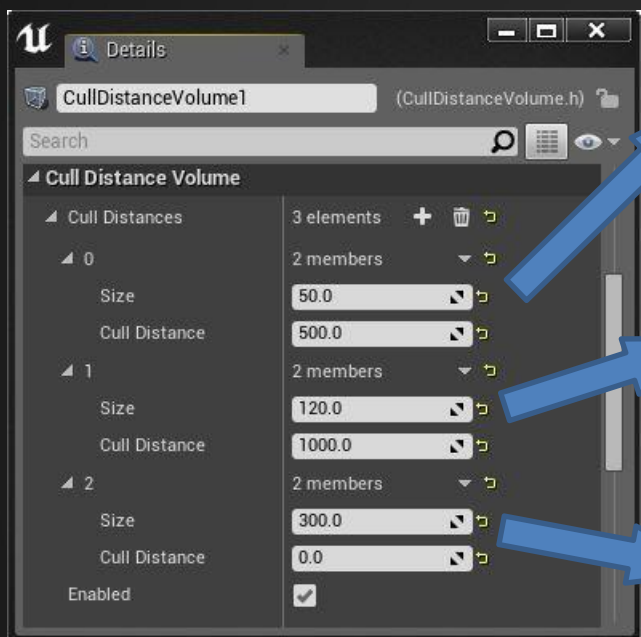
## 距离剔除(Distance Culling)的工具与使用方法

- Cull Distance Volume工具，把大于相距相机一定距离并且小于一定尺寸的对象给剔除掉。



# 瓶颈处理 · 剔除 (Culling)

## 距离剔除(Distance Culling)的工具与使用方法



- 在该Volume内的对象中，如果它们的尺寸小于50个单位，那么当这些对象距离相机500个单位或者更远时，将会剔除这些对象。
- 在该Volume内的对象，如果它们的尺寸小于120个单位，那么当这些对象距离相机1000个单位或者更远时，将会剔除这些对象。
- 在该Volume内的对象，如果他们的尺寸大于300个单位，那么将 **永远不会剔除** 该对象，因为在这种情况下0代表无穷远，该相机永远都不会达到那么远。当你想要体积内尺寸大于一定数值的对象永远不被剔除，可以使用这个设置。

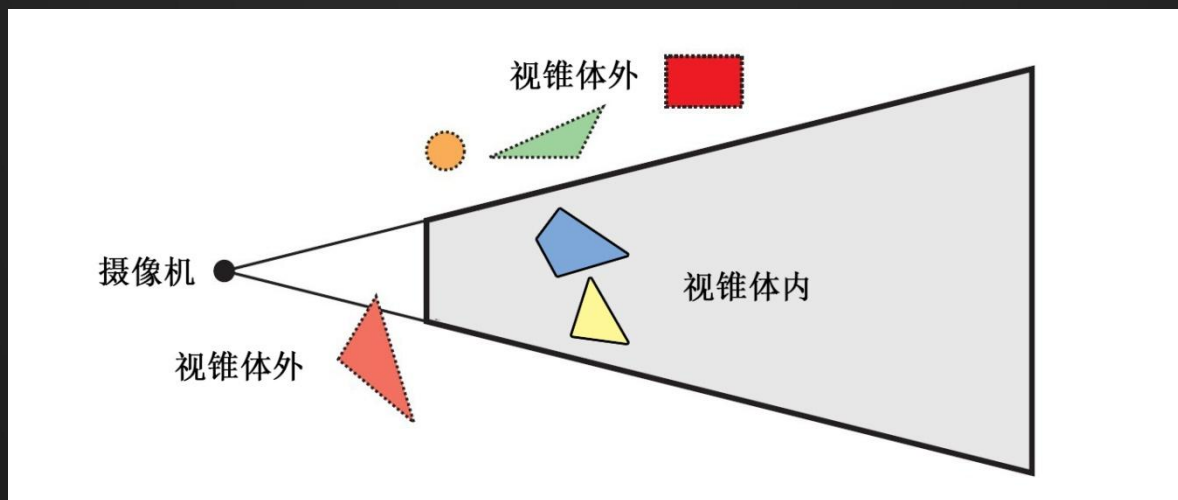
\*Size指的是该物体最长边的长度。

比如说一个BOX的尺寸是50×100×120，那么这个BOX的Size就是120。

# 瓶颈处理 · 剔除 (Culling)

## 视锥体剔除(Frustum Culling)

- 将摄像机的视野(field of view)以外的物体给排除掉，被排除掉的物体不会进入渲染的几何阶段(Geometry Stage)。
- 通俗点讲就是屏幕以外的东西不会被渲染，大大减少了渲染消耗。



# 瓶颈处理 · 剔除 (Culling)

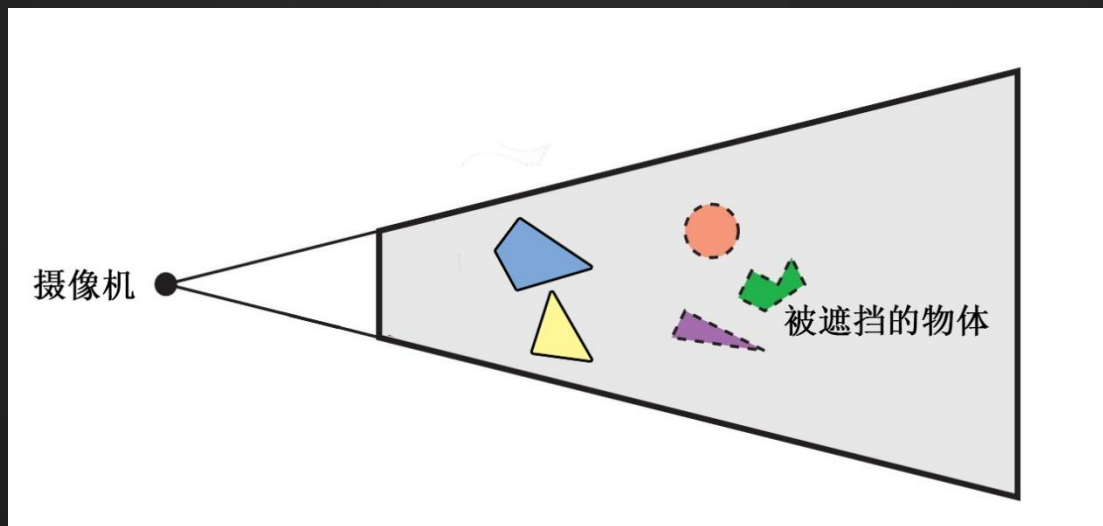
针对视锥体剔除(Frustum Culling)的优化方案

- 将画面变得精简，在同一屏幕内同时出现的物体尽量减少。
- 将关卡内的物体给分散，这样出现在同一屏幕内出现的物体会减少。

# 瓶颈处理 · 遮挡剔除 (Occlusion Culling)

## 遮挡剔除(Occlusion Culling)

- 根据场景中物体的位置预先生成场景Occlusion Culling数据，运行时就可以剔除对应物体之后的其他物体。遮挡剔除是减少渲染消耗的有效手段之一，可以和视锥体剔除同时生效，进一步减少渲染的消耗。





# 瓶颈处理 · 遮挡剔除 (Occlusion Culling)

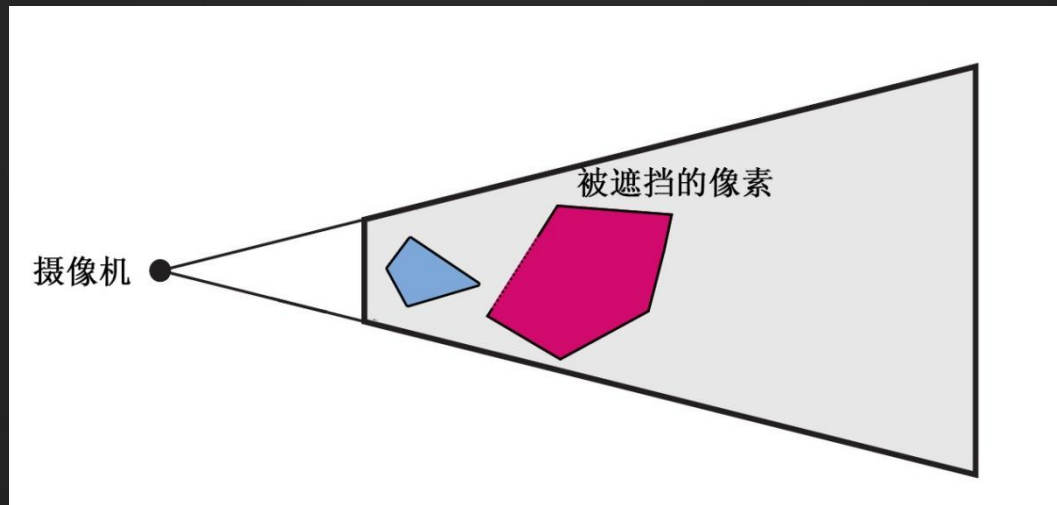
针对遮挡剔除(Occlusion Culling)的优化方案

- 当同一屏幕内同时出现大量的物体，可以用单个的大型物体挡住，比如说一面大型的墙。

# 瓶颈处理 · Early Z Pass

## Early Z Pass

- 遮挡剔除完成后，还是有大量被遮挡的像素将被绘制，这也会带来不必要的开销。于是此处做Early Z Pass。
- 类似于遮挡剔除物体，这次我们剔除掉被遮挡的像素。



# 瓶颈处理 · Early Z Pass

## Early Z Pass

- 重点来了!!!!

半透材质物体(Translucent)为了可以和已有像素混合, 是不能开启深度写入的, 所以:

**半透材质是不参与此步骤的!**

**半透材质是不参与此步骤的!**

**半透材质是不参与此步骤的!**

因此造成了我们常说的**重复绘制(Over Draw)**现象, 既要渲染半透材质物体, 还要渲染后面未被剔除的像素, 渲染次数以倍数增长。

# 瓶颈处理 · 重复绘制(Over Draw)

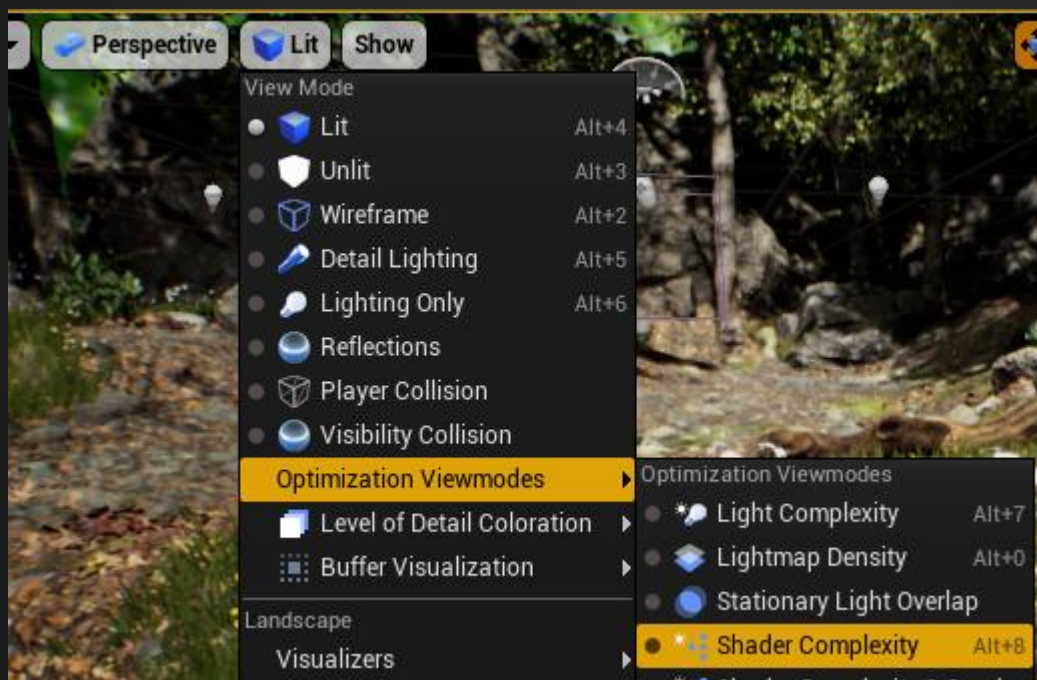
针对重复绘制(Over Draw)的优化方案

- 尽量减少半透材质占据屏幕的面积，宁愿增加模型顶点数来代替半透材质的使用。
- 透贴材质(Mask)也是如此，虽然它不是混合像素，但是它是像素先全部绘制后，再进行mask，把没用的像素扣掉，再绘制后面的物体，所以被扣掉的部分也是重复绘制区域。
- 延迟贴花(Deferred Decal)本质上也是半透混合模式，所以它也会造成重复绘制，大面积的延迟贴花使用Plane模型代替，或者使用顶点绘制。

# 瓶颈处理 · 重复绘制(Over Draw)

## 重复绘制(Over Draw)检查工具

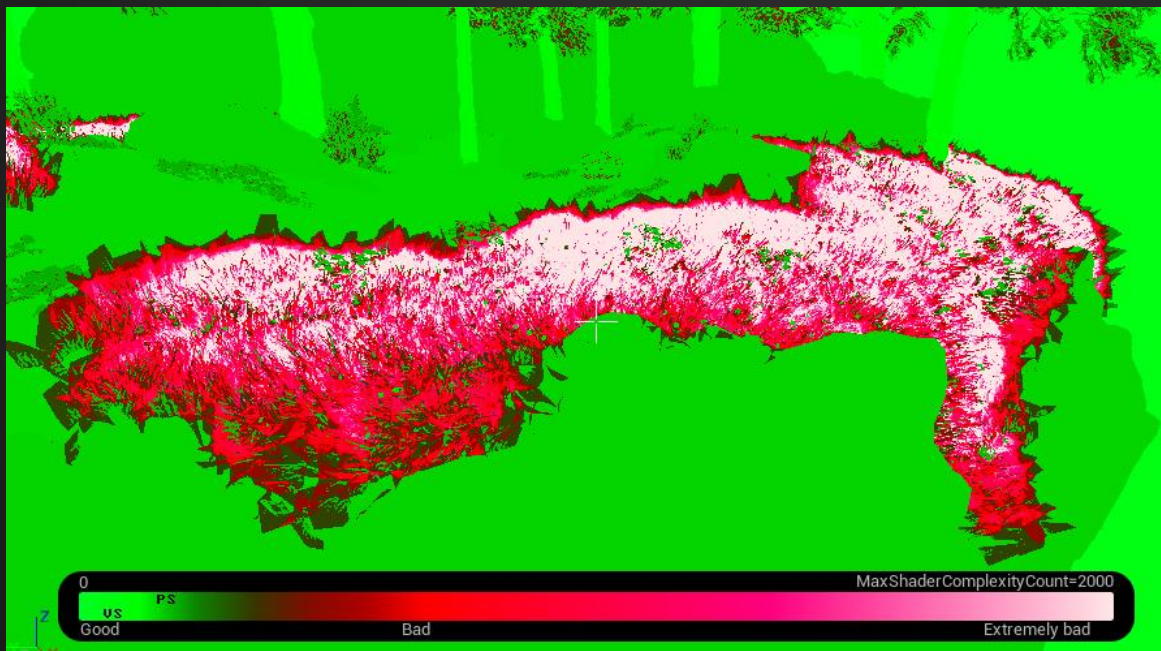
- View Mode下的Shader Complexity工具，快捷键 Alt+8，运行的时候快捷键是F5。



# 瓶颈处理 · 重复绘制(Over Draw)

## 重复绘制(Over Draw)检查工具

- 颜色条由绿色向粉白色过度，绿色是最理想的，只绘制一次，粉白色十分糟糕，绘制了多次，如下图，草地因为多次的重复绘制，形成恐怖的粉白色。





# 瓶颈处理 · 重复绘制(Over Draw)

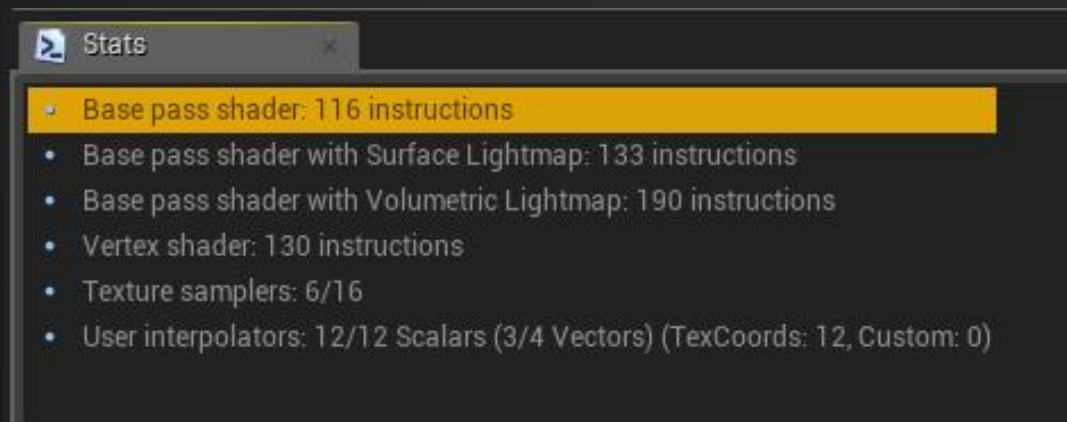
## 针对大片草地的优化方案

- 我们制作草地的通常做法是使用几张Plane模型穿插，然后使用Mask贴图形成草的样子，这样的话就造成了我们之前说的重复绘制问题,而且草地是由N颗草重叠构成，绘制的重复次数更是N倍。
- 所以一个解决方案是细化草模型，减少透贴面积。
- 但是这样的话模型顶点数也会成倍增加，同样造成不小的压力，所以最后综合考虑，使用LOD的思路，屏幕近处的草细化顶点，屏幕远处的草因为占据屏幕的面积减少，重复绘制面积就会减少，就可以使用Plane模型了。

# 瓶颈处理 · 材质(Material)

## 材质的绘制指令(Instruction)

- 材质的Instruction指的是材质所在的模型的每个被渲染的像素每帧需要的绘制指令。举个例子，一个材质的指令有100条，这个材质所在的模型最终被渲染的像素有500个，那最后GPU每帧需要执行的指令是 $100 \times 500$ 条，也就是50000条。*(从这里看出，GPU的运算能力是多么强大，也可以从侧面认识到优化Draw Call的重要性)*





# 瓶颈处理 · 材质(Material)

## 针对Instruction的优化

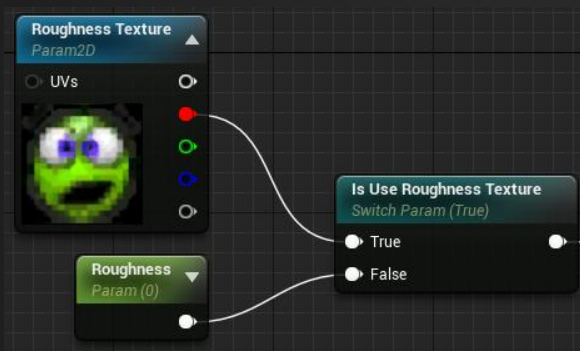
- 要时刻关注Instruction的数量。
- 只有Apply之后才是最终的Instruction。
- Epic官方自研游戏Paragon的制作标准：  
角色最多350 Instruction  
环境模型一般150-200，上限255
- 因为Instruction是根据每帧每像素进行计算，所以根据材质对应的模型大小来具体决定Instruction的数量。



# 瓶颈处理 · 材质(Material)

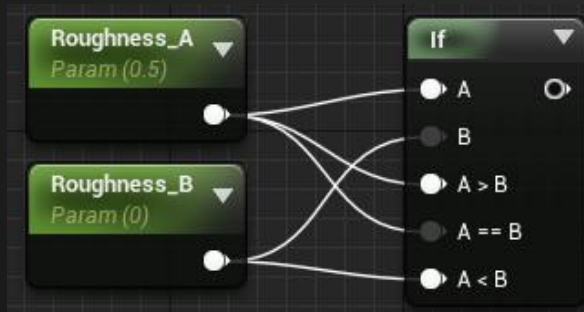
## 针对Instruction的优化

- 如果材质比较复杂，可以使用Static Switch Parameter关掉不需要的运算。



在计算Static Switch Parameter的时候只会去计算True跟False其中一条线上的逻辑，不会浪费运算量。

- 相对的，少用If命令。

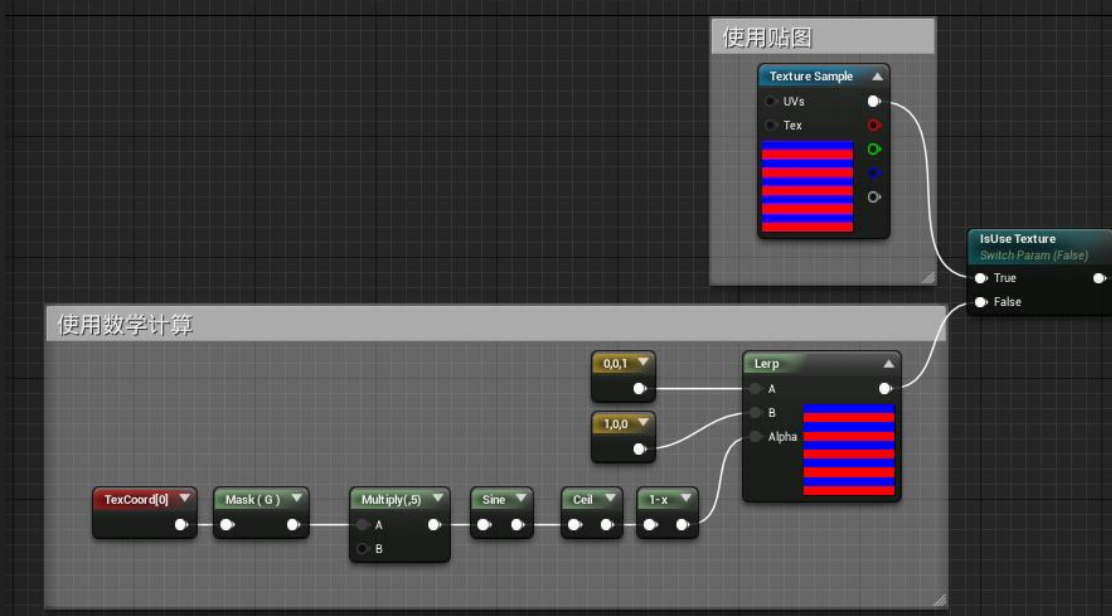


在计算If的时候A跟B要实时去比较计算来得到到底是使用A还是B，也就是说，A跟B上的逻辑都要计算。

# 瓶颈处理 · 材质(Material)

## 针对Instruction的优化

- 不要使用太复杂的数学计算，可以使用合理的的纹理贴图替换复杂的数学计算。举个例子：



同样是蓝红条纹的效果，使用贴图的Instruction比使用数学运算的Instruction少了5个。（当然，上例的数学运算十分简单，Instruction差距并不大）

# 瓶颈处理 · 材质(Material)

## 针对Instruction的优化

- 充分利用贴图的R/G/B/A四个通道，把四张不同的灰度图分别放在四个通道中合并为一张贴图，这样既可以节省Instruction，还可以节省内存。

# 瓶颈处理 · 灯光(Lighting)

## 灯光 (Lighting)

- UE4中灯光分为三种属性，分别为静态光(Static)、固定光(Stationary)、动态光(Movable)。
- 性能消耗：静态光消耗最低，固定光其次，动态光消耗最大。



本文件只简单讲一下基本优化方案，详细的灯光属性、性能以及具体操作放在另外的文件中。

# 瓶颈处理 · 灯光(Lighting)

针对灯光 (Lighting) 的优化方案

- 动态光的开销极大，尽量小范围使用，减少动态光影响范围。
- 定向光源(Directional Light)，也就是所谓的太阳光，只要没有位置以及角度变化，建议使用固定光。
- 减少灯光（动态光，固定光）的重复覆盖区域，其中固定光的覆盖区域最多只允许覆盖四次，第五个灯光会被当作动态光处理，因为固定光需要占据一个通道，而通道只有R/G/B/A四个。

(需要注意的是，如果太阳光也是固定光的话，那另外只允许重复三个固定灯，因为，太阳光也占据了一个通道)

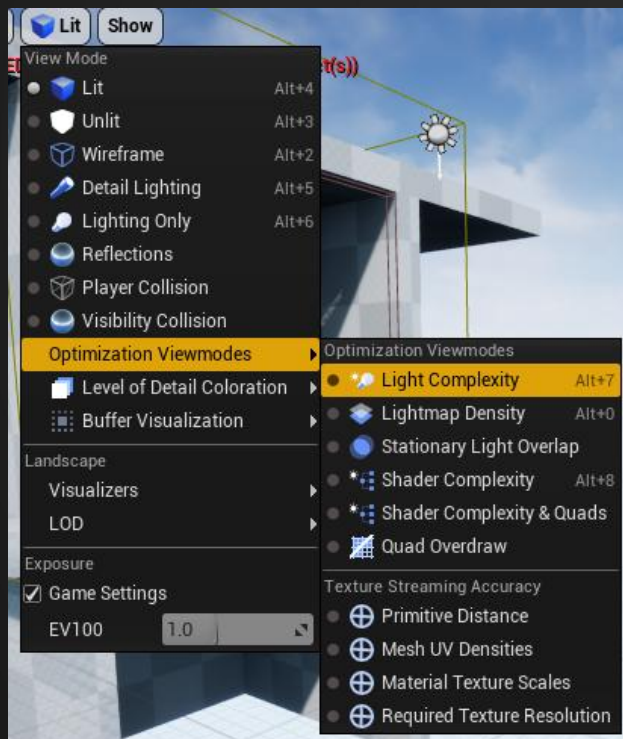


# 瓶颈处理 · 灯光(Lighting)

针对灯光 (Lighting) 的优化方案

- 灯光重复覆盖的检查工具

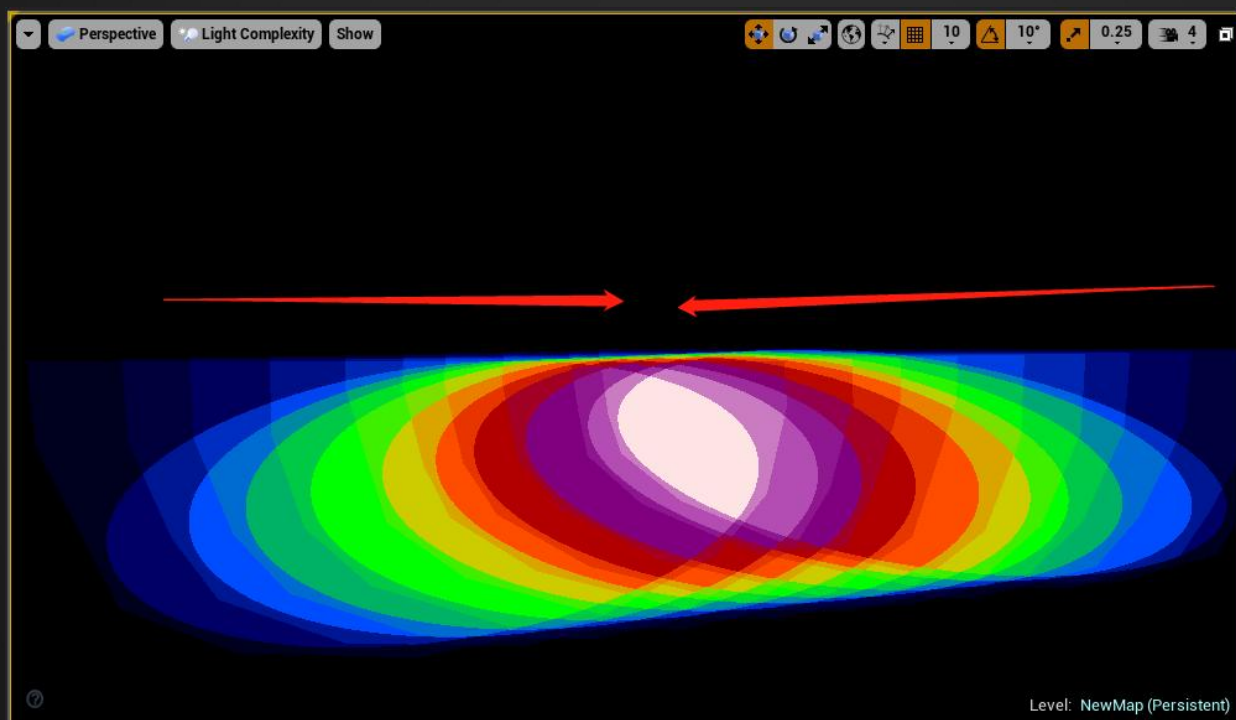
View Mode下的Light Complexity工具， 快捷键Alt+7。



# 瓶颈处理 · 灯光(Lighting)

针对灯光 (Lighting) 的优化方案

- 灯光重复覆盖的检查工具  
灯光重复覆盖程度由冷色到暖色过渡。

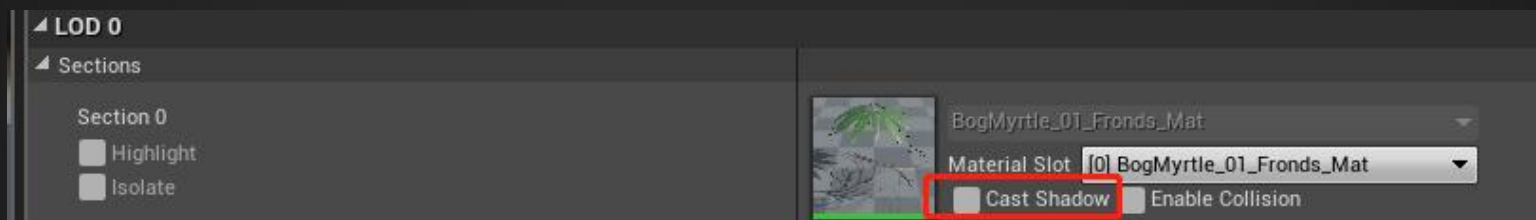




# 瓶颈处理 · 灯光(Lighting)

针对灯光 (Lighting) 的优化方案

- 减少动态阴影的开启，有些看不见阴影的物体就关闭阴影，比如说地板，贴花。UE4中动态阴影是很伤的。
- 离屏幕远的看不见阴影的物体也可以通过LOD关闭阴影。



- 静态构建的开销很低，能静态构建就尽量静态构建。